PlaceMux Matching Engine: End-to-End Demo

This notebook demonstrates the end-to-end flow of the matching engine: Data loading -> Feature Engineering -> Matching -> Ranking -> Explainability.

In [1]:
```
import os, sys
import pandas as pd
sys.path.append(os.path.abspath(".."))
from src.feature_engineering import get_feature_spaces
from src.matcher import calculate_match
from src.explainability import generate_reasons
from src.ranking import rank_candidates
from src.metrics import simulate_ground_truth_and_evaluate
```

1. Data Loading and Feature Engineering

In [2]:
```
students_df, jobs_df = get_feature_spaces("../data/students.csv", "../data/jobs.
display(students_df.head())
display(jobs_df.head())
```

	Student ID	Name	Verified Python Score	Verified SQL Score	Verified ML Score	Communication Score	Aptitude Score	Project Count	Int
0	1	Uma Davis	78	91	68	64	92	4	
1	2	Alice Anderson	58	62	50	60	73	4	
2	3	Hannah Taylor	42	61	92	51	73	3	
3	4	Evan Davis	99	60	72	61	71	4	
4	5	Victor Clark	66	98	81	77	65	5	

	Job ID	Company Name	Role	Required Skills	Minimum Skill Scores	Minimum CGPA	Experience Requirement	R
0	1001	Startup_10	Research Scientist	ML,Python	60,63	7.9	2	
1	1002	Corp_8	ML Engineer	Python,ML	67,82	7.5	1	
2	1003	TechCompany_13	Research Scientist	Python,ML,SQL	73,63,89	6.5	2	'I
3	1004	TechCompany_10	ML Engineer	Python,ML	62,67	6.7	3	
4	1005	TechCompany_4	Data Scientist	SQL,ML	72,72	7.9	0	

2. Match Single Student to Single Job

In [3]:

```
student = students_df.iloc[0].to_dict()
job = jobs_df.iloc[0].to_dict()
print(f'Student: {student["Name"]}')
print(f'Job: {job["Role"]} at {job["Company Name"]}')

score, details = calculate_match(student, job)
reasons = generate_reasons(score, details)

print(f"\nMatch Score: {score}%")
print("\nReasons:")
for r in reasons:
    print(r)
```

```
Student: Uma Davis
Job: Research Scientist at Startup_10

Match Score: 87%

Reasons:
🟢 ML score (68) exceeds requirement (60) by 8 points.
🟢 Python score (78) exceeds requirement (63) by 15 points.
🔴 CGPA (5.8) is below the minimum requirement of 7.9.
🟢 Experience requirement met with 0 internship(s) and 4 project(s).
🟡 Communication skills (64) could be improved.
```

3. Top-N Ranking

In [4]:

```
job = jobs_df.iloc[0].to_dict()
print(f'Ranking for Job: {job["Role"]} at {job["Company Name"]}')

ranked = rank_candidates(job, students_df, top_n=3)
for rank, candidate in enumerate(ranked, 1):
    print(f'\nRank {rank}: {candidate["student_name"]} (Score: {candidate["match
    for r in candidate["reasons"]:
        print(f"  {r}")
```

```
Ranking for Job: Research Scientist at Startup_10

Rank 1: Alice Brown (Score: 100%)
  🔴 ML score (59) falls short of requirement (60) by 1 points.
  🟢 Python score (92) exceeds requirement (63) by 29 points.
  🟢 CGPA (8.0) satisfies the minimum requirement of 7.9.
  🟢 Experience requirement met with 2 internship(s) and 3 project(s).

Rank 2: Wendy Martin (Score: 100%)
  🟢 ML score (94) exceeds requirement (60) by 34 points.
  🟢 Python score (72) exceeds requirement (63) by 9 points.
  🟢 CGPA (8.0) satisfies the minimum requirement of 7.9.
  🟢 Experience requirement met with 2 internship(s) and 4 project(s).

Rank 3: Xavier Garcia (Score: 100%)
  🟢 ML score (76) exceeds requirement (60) by 16 points.
  🟢 Python score (66) exceeds requirement (63) by 3 points.
  🟢 CGPA (9.7) satisfies the minimum requirement of 7.9.
  🟢 Experience requirement met with 3 internship(s) and 3 project(s).
  🟢 Strong communication skills (82).
```

4. Evaluation Metrics

Note on Ground Truth: In this evaluation, a "true match" is defined strictly using an independent, rule-based approach. A student is considered a true match if they meet *all* of the job's minimum requirements: their average score for the required skills must meet or exceed the job's average minimum skill requirement, their CGPA must be at least the job's minimum CGPA, and their total experience units must be greater than or equal to the job's experience requirement.

In [5]:

```python
metrics = simulate_ground_truth_and_evaluate(calculate_match, students_df, jobs_
print("\nEvaluation Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

```
Ground Truth Positive Rate: 13.63% (6817/50000 true matches)
Logged experiment: baseline_rule_based - {'precision': 0.17844615465158892, 'reca
ll': 1.0, 'fpr': 0.7267906352036682, 'accuracy': 0.3723}

Evaluation Metrics:
precision: 0.1784
recall: 1.0000
fpr: 0.7268
accuracy: 0.3723
```